

ZipRecruiter Interview Debrief - October 7, 2026

Rounds 2 and 3, following the October 6 screen with Scott (product leader). Full transcript scrubbed the same day.

Round 1: Steve - Coding / debugging (Go, CodeSignal)

Interviewer: Steve, engineer on the S&B (small-to-medium business) team, 5+ years at ZipRecruiter, LA area. His team owns job hosting and management flows: full-stack job management services plus underlying marketplace mechanics.

Format: A ~200-line Go CLI program - a self-service IT tool for managing hardware features attached to laptops. A bug report from a user ("George"): the update laptop features flow throws an error in the "this doesn't work" scenario but not in the "this works" scenario. Task: reproduce, find the bug, fix it. No Go background required.

How it went:

- Reproduced first, before theorizing. Ran the working scenario, then the broken one, and confirmed the error.
- Followed the stack trace to `insertFeaturesIntoDB` and identified the composite unique constraint on (username, laptop, feature).
- Root-cause hypothesis, stated clearly: the update path re-inserts rows that already exist instead of computing a delta, so the second loop iteration trips the unique constraint.
- Proposed fix: check whether the (user, laptop, feature) row already exists before inserting; skip the insert if it does. Noted the existing select-all statement could serve as the check.

What went well:

- Systematic debugging under an unfamiliar language: reproduce, read the trace, localize, hypothesize, propose a minimal fix.
- Asked good clarifying questions early (what the tool is for, which scenario should fail).
- Handled the environment friction (network issues, hotspot switch) without losing composure.

What went wrong / gaps:

- Needed Steve's redirect to understand it was a multi-step flow, not a single broken command; burned early minutes on the wrong laptop (M vs M2).
- Never implemented the fix. Time went into scaffolding new helper functions instead of the minimal guard clause. In a 45-minute exercise, the working change matters more than the design of the change.
- Asked Steve "how would you solve it" at the end - he declined (he administers the question often). Fair to ask, but have a tighter close ready: restate your diagnosis and fix in two sentences and stop.

The ideal fix, for the record: in the update path, before `insertFeaturesIntoDB`, check existence of the (user, laptop, feature) row and skip the insert when present - or delete-then-insert the full set. A guard clause, not new functions.

Intel from Steve:

- ZipRecruiter's stack: TypeScript is the most common language, Go is number two - chosen in a "cookoff" about 6-7 years ago for async processing, fan-out, and performance.
- AI adoption is aggressive: Claude Code used heavily, "almost nothing that I'm putting in a PR is actually handwritten anymore," AI-reviewed PRs. The corresponding investment is in

Round 2: Yifan - System design (third-party job ingestion)

Interviewer: Yifan, 2+ years at ZipRecruiter. Previously managed the activity data and activity-understanding team (job-seeker interaction signals - clicks, applies, impressions - turned into embeddings feeding recommendation systems). Now on paid acquisition: balancing the marketplace by spending efficiently against job-seeker long-term value.

Problem: ZipRecruiter receives first-party postings (employers uploading through the SMB portal). Now ingest third-party postings too - crawlers (per site, per region) and XML job feeds - into one unified job store powering analytics/BI, ML training, user-facing apps, and event-based apps. Scale: ~20M raw drops/day, ~500K unique jobs/day. Freshness bar: one day of staleness acceptable.

The design presented:

1. Per-source/per-region crawlers plus the existing first-party upload path.
2. A Kafka-style queue in front of everything, with per-topic dedup so the store never sees the same drop twice.
3. A data-unification plane: reads topics, flattens every format into one common schema (JSON), rejects bad data.
4. Storage split by read pattern: Redshift-style structured store for analytics/BI, S3 for ML training, a DynamoDB-style metadata store (source ID, crawl frequency, trust level) feeding the crawlers, and a key-value job store behind a gateway for user-facing APIs and SQS/SNS event delivery.

The dedup deep dive (where the round spent most of its time):

- Natural key: the job ID assigned by the employer, plus a ZipRecruiter-internal ID so the system never depends on an external identifier it doesn't control.
- A master job-ID table for millisecond existence checks, storing a hash of the job description so freshness is a hash comparison, not a re-parse.
- A job-source table recording which sources (LinkedIn, Indeed, employer direct) carry the job.
- Field-level merge with authoritative-source ranking: for each field the system wants populated, sources are weighted - the employer itself outranks aggregators (e.g., conflicting pay ranges resolve to the employer's number).
- A daily cron per source comparing yesterday's hashes to today's; only changed jobs flow through. First-party uploads use a push variant of the same date-created filtering.

What went well:

- Clarified scale, freshness, and read/write patterns before drawing a single box.
- Full end-to-end flow, then went deep where asked instead of staying shallow everywhere.
- Concrete schema thinking under dedup pressure - keys, hashes, tables, merge rules.
- Strong close: asked about agentic crawling and how the architecture changes when AI agents (not humans) read the listings. Yifan confirmed sister teams already run agent-based validation on crawled jobs. Good instinct to end on where the industry is going.

What to tighten:

- Long monologues. He caught it himself ("I went in multiple directions") - the fix is to pause for a breath and a check-in every 90 seconds: "does this match what you're thinking, or should I go deeper somewhere?"
- Named the gaps only at the end (failure modes, training-data bias from source skew). Name them during the design: "the two things I haven't covered are failure handling and bias - want me to take one?"
- Less whiteboard fiddling (colors, arrows); the boxes matter less than the decisions between

them.

Reusable STAR stories banked from this transcript

Story A: The 400K PDFs - navigating the AI-vs-determinism conflict

Situation. Moderna holds 400,000+ paper documents as PDFs, 8-12 pages on average - regulatory documents written for humans. The goal: digitize them into queryable knowledge (parse, chunk, index).

Task. Two camps in open tension. One: use no AI at all - these are regulatory documents, traceability is non-negotiable. The other: use AI for everything. Shridhar's own bias is toward heavy AI use.

Action. Refused to argue positions; built a mechanism instead. Started from the company objective both sides share - do not harm patients - trickled it down to org goals (save money, be efficient), and is building a per-situation risk-score framework that says, for a given document type and use, what level of AI is acceptable. Both parties are co-authoring it and it gets revised as they learn. Explicit principle: separate the people from the issue.

Result. Still in progress - frame it as "how I'm handling it," not "solved." The mechanism depersonalized the conflict and both sides are contributing.

Plays for: "Tell me about a time you navigated competing stakeholders," influence without authority, disagreement between teams, introducing AI into a regulated environment.

Story B: The LLM log-mining app (from the Steve round)

Situation. Brought into Moderna to look at 40+ software applications - in-house code stitched with off-the-shelf tools running end-to-end manufacturing workflows. Mandate: figure out what to deprecate, what to enhance, save money and time.

Action. First build: a telemetry pipeline. Got the apps emitting logs to S3 (some already were), built Redshift dashboards, and put a thin vibe-coded layer on top calling the OpenAI API - it queries the logs for questions users ask repeatedly and surfaces them as common features, cutting per-query license fees on usage-priced tools. Constraint-driven throughout: no new software, no new spend, months-long security approvals avoided by reusing S3/Redshift; unstructured blobs (JSON, sometimes literal PDFs) normalized by a thin conversion layer.

Hindsight (his own). Would separate system metrics, business metrics, and usage data into different streams with different retention (CloudWatch-style for 24-hour infra health, S3 for the long tail); would add an SQS-style queue with priority tiers instead of treating every user request as equal; would build model-agnostic interfaces (an OpenAI format change broke tightly-coupled code); and would have pushed back harder on the business to balance short-term AI demos against the tech debt they created.

Plays for: "Tell me about a technical decision you'd revisit," cost optimization, constraint-driven architecture, managing tech debt, build-vs-buy.

Interview intel (new)

- ZipRecruiter eng culture: TypeScript first, Go second; heavy Claude Code adoption with AI-reviewed PRs; investing in testing/monitoring/rollback to match.
- Sister teams already run agent-based validation over crawled jobs before they hit unified

storage - the agentic direction is real inside the company.

- Both interviewers were engineers close to the work (S&B marketplace mechanics; activity-data/paid acquisition) - the loop is weighting hands-on depth.

Personal notes surfaced in the transcript

- Shridhar gave three reasons for the move, crisply: (1) the next stage at Moderna means going deeper into biology, which is not his interest - he wants core technology companies where tech is the bread and butter; (2) his wife's residency (Harvard Medical) ends next year and they plan to move to Seattle, where Moderna's presence is small; (3) the MIT MBA exposed him to business/finance/product and he wants to apply it.
- Keep the three-reason structure. It landed well ("those 3 are all good reasons").
- The October 6 coaching still applies: "I have reached the ceiling of what computer-science knowledge alone can do in biotech" beats "getting bored," and the metrics fluency stays the differentiator.

Addendum (2026-10-07): areas of improvement + the ideal crawler design

Areas of improvement from the Yifan round

1. **Monologue length.** Several stretches ran minutes without a check-in ("I went in multiple directions" - his own words). Fix: pause roughly every 90 seconds - "does this match what you're thinking, or should I go deeper somewhere?"
2. **Failure modes and training-data bias surfaced only in the last two minutes.** Name them during the design: "two things I haven't covered are failure handling and source-skew bias - want me to take one?"
3. **The crawler was the weakest subsystem.** Politeness, change detection, and dedup-at-the-edge were hand-wavy; fetch and parse were coupled with no replay story. Reworked below.
4. **Whiteboard mechanics over decisions.** Time went to colors and arrow directions; the decisions between the boxes matter more than the boxes.

Ideal solution: the crawler subsystem, done properly

Requirements, stated up front. The crawler is a fetch-only tier: raw bytes plus fetch metadata into immutable storage. Parsing, extraction, and normalization happen downstream. One-day staleness is acceptable. Hard constraints: never violate a domain's politeness budget (robots.txt, crawl-delay, 429 handling), never fetch the same URL twice concurrently, never re-download what hasn't changed.

Architecture.

1. **Source config store** (per source: seed URLs, sitemap URLs, robots rules, crawl budget, trust tier, expected change rate). This part of the in-round design was right; keep it.
2. **Frontier.** A priority queue of URLs to fetch. Priority = source trust × business value × staleness × expected change rate. Sharded per domain so one slow domain never blocks the rest; per-domain token buckets enforce crawl-delay and concurrent-connection caps.
3. **Fetcher fleet.** Stateless, horizontally scaled workers. Every request is a conditional GET (ETag / If-Modified-Since): an unchanged page costs a 304, not a download. robots.txt cached per domain with a TTL; crawl-delay honored; 429/503 responses honor Retry-After. Timeouts with exponential backoff; a per-domain circuit breaker trips after sustained failures and routes to a dead-letter queue with alerting. The user-agent identifies the crawler with a contact address.
4. **Seen-URL set.** A Bloom filter sized for ~20M URLs/day: at 1% false-positive rate that's about

24MB and 7 hash functions - small enough to hold in memory. The tradeoff to state out loud: a false positive means skipping a genuinely new URL. Mitigation: sitemap-driven discovery catches it the next day, or rebuild a counting Bloom daily. Say the tradeoff; don't hide it.

5. **Change detection at the edge.** Simhash the extracted text (64-bit fingerprints over shingles); Hamming distance of 3 or less means boilerplate churn, not a real change. Only meaningfully-changed documents flow to the unification plane. This moves the 20M-to-500K funnel to the edge, before paying for parsing.
6. **Raw store.** S3, content-addressed (key = content hash). Identical bytes fetched twice store once: free dedup at the storage layer. Immutability is the replay story - when the parser has a bug, re-parse from S3; never re-crawl.
7. **Sitemap-first discovery.** Where a source publishes a sitemap, diff yesterday's `<urlset>` against today's using `<lastmod>` instead of crawling listing pages. This formalizes the "hash from yesterday" idea from the round and is an order of magnitude cheaper than brute-force crawling.
8. **XML job feeds: separate fast path.** Already structured: validate against the schema, straight into the queue. No crawler involved.
9. **Adaptive scheduling (the elegant touch).** Estimate each URL's change rate as a Poisson process from its history; set the re-crawl interval proportional to $1/\lambda$. Fast-changing postings get crawled often, stable ones rarely - the schedule learns instead of being fixed.

Scale math (say it in the room). 20M fetches/day is ~230/sec average, ~700/sec at 3× peak. At ~50KB per page that's roughly 1TB/day of raw ingress - but conditional GETs, simhash filtering, and content-addressed storage mean only changed documents persist downstream. At ~2 seconds per fetch under politeness limits, peak needs ~1,400 concurrent fetch slots: about 15-20 modest workers. Small fleet; the complexity is in scheduling, not metal.

Failure handling (the self-flagged gap, closed). Domain down: circuit breaker, backoff, DLQ, alert on sustained failure. Poison URLs (calendar traps, session-ID explosions): URL canonicalization plus per-domain URL-count caps. Parser bug: immutable raw store, re-parse without re-crawling. Stale robots.txt: refresh on TTL, fail closed (don't crawl) when refresh fails. Frontier depth is the autoscaling signal for the fetcher fleet.

The 30-second talk track. "The crawler is fetch-only; parsing is downstream and replayable. Discovery is sitemap-first, crawling second. Conditional GETs and simhash at the edge move the 20M-to-500K funnel before we pay for parsing. Per-domain politeness budgets and circuit breakers keep us unblocked and a good citizen. Scheduling adapts to each page's observed change rate."

Transcription artifacts corrected while scrubbing: "Shredder" = Shridhar, "commuter science" = computer science, "Steve"/"Yifan" as heard.